

AI 활용 기술 문서

LATENT — 레벨을 미리 만들지 않는 퍼즐

저장소 github.com/TaraGamesDev/latent-nan2026 참가 개인

1. 한 문장

캐주얼 퍼즐의 원가는 레벨뱅크이고, 그 원가에서 비싼 쪽은 만드는 일이 아니라 만든 레벨이 풀리는지 검증하는 일입니다. 레벨을 미리 만들지 않으면 그 검증 단계가 통째로 사라집니다.

LATENT의 나사는 판에 덮여 있는 동안 색이 없습니다. 관측되지 않았으니 아직 존재할 이유가 없습니다. 위의 판이 떨어져 손이 닿는 순간, AI가 플레이어의 그 시점 상황을 읽고 색을 씁니다 — 그때 “이 벽은 끝까지 해체 가능한가”를 부등식 하나로 보장하면서. 사후 검증이 필요 없습니다. 애초에 불가능한 상태를 만들지 않기 때문입니다.

2. 왜 이 방향인가 — 비싼 것은 생성이 아니라 검증

소코반 레벨 자동 생성은 논문 주제입니다. MCTS로 생성하고, 난이도는 “문제 분해 지표”나 “사람의 탐색을 흉내낸 계산 모델”로 추정합니다. 이유는 단순합니다 — 소코반은 PSPACE-complete라 만든 레벨이 풀리는지, 얼마나 어려운지를 싸게 알 방법이 없습니다. 그래서 사람이 검수해야 하고, 그게 원가입니다.

그러므로 장르 선택의 기준은 “재미있을까”가 아니라 “난이도와 풀림을 근사가 아니라 정확히, 싸게 알 수 있는가”가 됩니다. 이 축으로 후보 셋을 직접 구현해 재고 골랐습니다.

2.1 후보 비교 (측정값)

후보	검증 처리량	정확성
A · STAMP GF(2) 토글 퍼즐	85,228 레벨/초	완전 정확. 가우스 소거 한 번으로 풀림 여부·최소 수·해의 개수($2^{\text{영공간차원}}$)가 전부 나옵니다. 유일해 레벨은 풀리는 것의 37%.
B · MAGNET 당겨 붙이는 퍼즐	26 레벨/초	불완전. 레벨당 탐색 노드 28,988개. 깊이 6에서 6.7%만 풀리고, 나머지는 “안 풀린다”가 아니라 <i>모른다</i> 입니다.
C · 지연 결정 (채택)	470,000 결정/초	검증 단계가 없음. 배정 시점의 부등식으로 클리어 가능성이 구성상 보장됩니다.

B가 A보다 3,300배 느립니다. 이것이 이 프로젝트가 하려는 이야기의 축소판입니다. 소스는 `src/probe/`에 그대로 있고 `npx tsx src/probe/a-stamp.ts` 등으로 재현됩니다.

A는 “난이도를 정확히 계산한다”는 주장이 가장 강했지만 C를 골랐습니다. 이유는 검증 비용을 0으로 만드는 쪽이 더 근본적이기 때문이고, 시장에서 검증된 장르 위에서 그렇게 할 수 있기 때문입니다.

3. 아키텍처

나사에 손이 닿을 때마다

- [스킬 추정기] `src/ai/skill.ts`
`regret = value(솔버의 최선수) - value(플레이어가 고른 수)`
→ θ (숙련도), {계획성 · 정돈 · 템포}, {좌절도 · 지루함}
- [배정기 · 의도] `src/ai/assigner.ts`
 $\theta + \text{감정} + \text{벽 진행 램프} \rightarrow \text{목표 난이도}$
구제 확률 = (홀더 위험도 $\times$ w + 기본값) $\times$ (1 - 실력가중 $\times$ 목표)
후보: 즉시완성 / 홀더채우기 / 벽과같은색 / 새 색
- [배정기 · 시공] 같은 파일
★ 색 미결정 나사 수 ≥ 3 의 배수를 맞추는 데 필요한 나사 수
★ 지금 둘 수 있는 수가 하나도 없어지는 색은 쓰지 않는다
이 둘을 깨는 후보는 기각됩니다
- [글래스박스 패널] `src/ui/panel.ts`
위 전부를 화면에 그대로 노출

모듈	역할
<code>core/plates.ts</code>	벽 기하, 덮임 관계, 홀더 규칙
<code>core/rng.ts</code>	시드 기반 PRNG — 시드만 있으면 런 전체가 재현됩니다
<code>ai/solver.ts</code>	국면 평가와 수순 순위. 보드를 복제하지 않고 해석적으로 계산합니다
<code>ai/skill.ts</code>	수순당 후회 $\rightarrow$ 숙련도 추정, 감정 상태 분리
<code>ai/assigner.ts</code>	색 결정 + 완결성 검사 + 생존 바닥선
<code>ai/policy.ts</code>	정책 상수. 빌드에 구워짐
<code>ai/bots.ts</code>	봇 페르소나 — 콜드스타트 문제의 해법
<code>ui/scene.ts</code>	Three.js 씬. 전부 절차적 — 에셋 파일 없음
<code>tools/bench.ts</code>	단언하는 벤치. 보고하지 않고 실패시킵니다
<code>tools/margin.ts</code>	이 규칙에서 실력이 얼마나 값어치가 있는지 측정
<code>probe/</code>	2.1의 후보 비교를 재현하는 실측 코드

4. 핵심 설계 세 가지

4.1 수순당 후회(regret)로 실력을 잴다

“레벨을 클리어했는가”, “몇 분 걸렸는가”는 몇 분에 한 번 도착하고 그 레벨이 마침 쉬웠는지에 오염됩니다. 대신 모든 탭을, 그 탭이 이루어진 바로 그 국면에서 채점합니다.

$$\text{regret} = \frac{(\text{최선수의 가치} - \text{플레이어가 고른 수의 가치})}{\text{그 국면에서의 선택지 가치 편차}}$$

국면이 통제되어 있으므로 “쉬워서 잘한 것”과 “실제로 잘 둔 것”이 섞이지 않습니다. 선택지가 전부 동등한 국면은 **정보량 0**으로 처리해 추정을 흔들지 않고, 표본이 쌓일 때까지는 $\text{samples} / (\text{samples} + 10)$ 비율로 사전확률에서 벗어납니다 — 첫 판부터 최대 압박이 걸리지 않도록.

4.2 의도와 시공을 분리한다

배정기는 “지금 이 순간이 어떻게 느껴져야 하는가”만 정합니다. 실제로 색을 쓰기 전에 완결성 부등식이 **별도로** 검사합니다. **플레이어를 평가하는 쪽은 틀려도 되지만, 나사에 색을 쓰는 쪽은 틀리면 안 되기 때문입니다.** 추정이 어긋나면 난이도가 조금 안 맞을 뿐이지만, 배정이 잘못되면 그 벽은 영원히 해체할 수 없습니다.

4.3 약속은 작게, 대신 반드시 지킨다

불변식 — 이 벽은 끝까지 해체 가능하다.

모든 색은 최종적으로 3의 배수가 되어야 합니다. 그러지 못하면 남은 나사는 영원히 홀더를 비우지 못합니다. 그래서 매 배정마다 색 미결정 나사 수 $\geq$ 열려 있는 색을 닫는 데 필요한 나사 수 를 검사하고, 이를 깨는 후보는 기각합니다.

여기에 **생존 바닥선**이 하나 더 있습니다 — 색을 쓰는 그 순간에는 **둘 수 있는 수가 최소 하나** 존재해야 합니다. 이보다 강한 약속은 하지 않습니다. “해체 가능한 벽”과 “이 플레이어가 해체할 벽”은 다른 말이고, 후자를 약속하는 순간 게임이 사라지기 때문입니다.

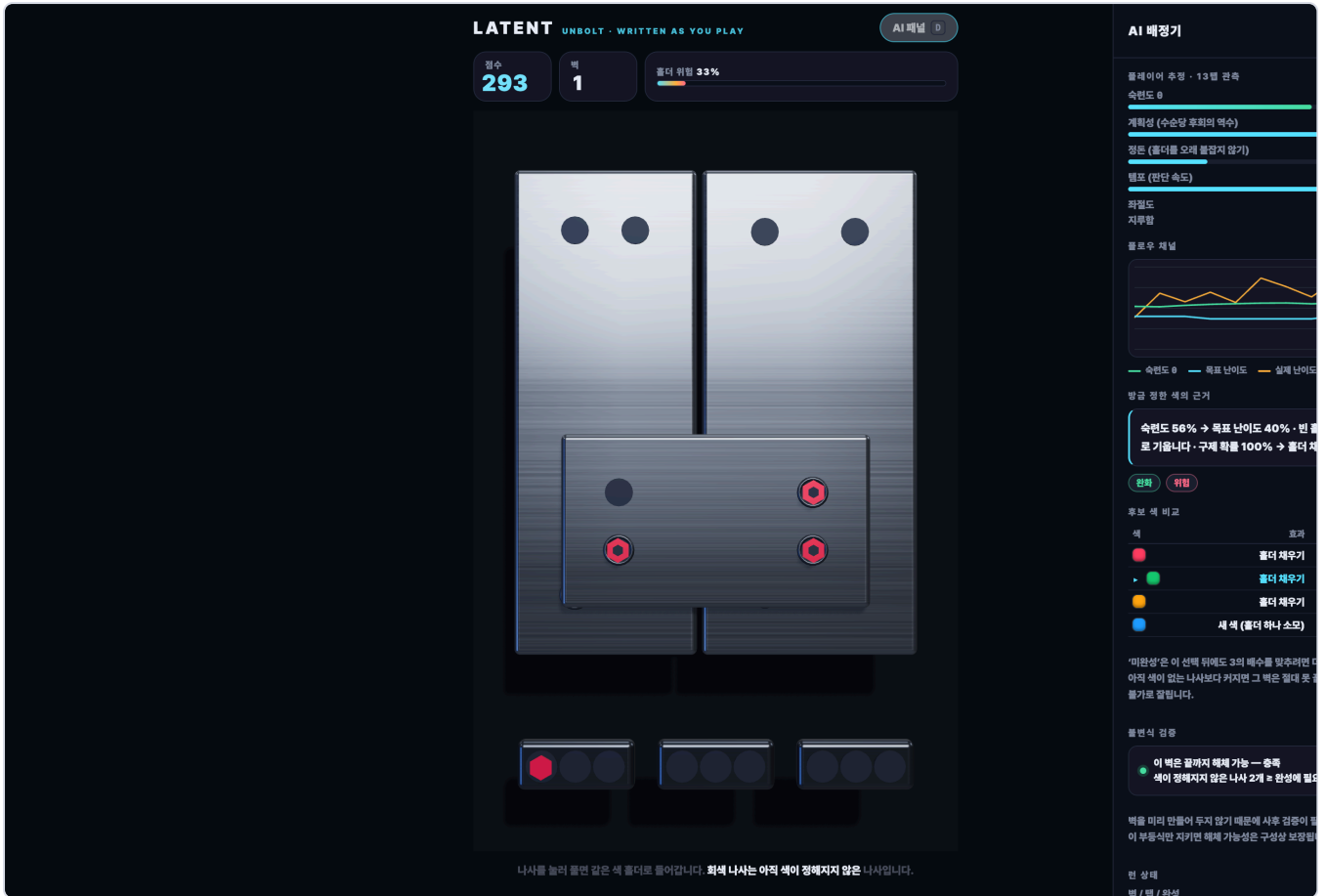
그리고 **색 폭 상한** — 손이 닿는 나사가 몇 개 안 될 때는 그 안의 서로 다른 색이 홀더 수를 넘지 않아야 합니다. 배포된 빌드를 실제로 플레이하다 **3탭 만에 패배**했고, 원인은 맨 위 판의 나사 네 개가 전부 다른 색이었던 것입니다. 홀더 셋 중 어느 셋을 잡아도 네 번째는 갈 곳이 없고, 그 판은 영원히 떨어지지 않습니다. 어려운 자리가 아니라 **불가능한 자리**였습니다.

두 제약은 동등하지 않습니다. 처음엔 하나의 필터에 함께 넣었더니, 둘 다 만족하는 후보가 없는 턴에서 폴백이 완결성 쪽을 어겼습니다 — 벤치 한 번에 301회. 완결성이 약속이고 색 폭 상한은 그 아래에서 양보하는 구성 규칙입니다.

다만 **질 수는 있습니다. 그래야 맞습니다.** 홀더가 3칸이므로 살아 있는 색이 3종을 넘는 순간 갈 곳 없는 색이 생깁니다. 실제로 **4종이 이 게임에서 사람이 막힐 수 있는 최소 팔레트**이고, 3종으로 묶으면 모든 색에 항상 자리가 있어 런이 끝나지 않습니다 — 그렇게 만들었더니 세 페르소나가 전부 3,000탭 상한까지 살아남았습니다. 배정기는 **색을 쓸 때만** 개입하고, 판이 떨어지기 전까지는 다시 쓰지 못합니다. 그래서 막히는 것은 **플레이어가 홀더를 잠근 순서의 결과**이지 배정의 실패가 아닙니다. 이 구분을 문서와 패널 양쪽에서 일부러 지켰습니다.

5. 글래스박스 — AI가 자기 판단을 보여준다

조용히 결정하는 시스템은 **자기 작업을 보여줄 수 있는 시스템보다 가치가 낮습니다.** 특히 이 장르는 플레이어들이 “게임이 나를 조작한다”고 의심하는 것으로 유명합니다. 그렇다면 숨길 것이 아니라 열어 두는 편이 낫습니다.



실제 플레이 중 캡처. 좌: 볼트로 고정된 철판 벽과 아래 홀더 3칸. 우: 추정치(속련도 0, 계획성·정돈·템포), 플로우 채널, 방금 정한 색의 근거, 그리고 후보 색 비교표 — 여기서 여러 후보가 완결성 ‘불가’로 기각된 것이 보입니다. 불변식이 실제로 후보를 자르고 있다는 증거입니다. 맨 아래에 불변식 검증 결과가 그대로 출력됩니다.

6. AI 도구 사용 내역

단계	사용 도구	내용
런타임 (배포 빌드)	없음	LLM 호출 없음. 게임 안의 “AI”는 전부 결정론적 코드 — 솔버, 추정기, 배정기. 네트워크·API 카·계정 불필요.
설계·구현	Claude Code (claude-opus-5)	코딩 협업 도구. 규칙 설계 토론, 엔진·AI·UI 구현, 헤드리스 계속 도구 작성, 설계 결함 진단. 6.2 참조.
에셋 생성	없음	이미지·사운드·모델·폰트를 생성하지도, 사용하지도 않았습니 다. 철판·나사·홀더는 코드가 만든 기하이고, 금속 질감의 브러시드 러프니스 맵과 환경광은 실행 시점에 생성합니다.

6.1 런타임에 LLM을 쓰지 않은 이유

- 심사자가 링크만 눌러도 돌아가야 합니다. 사전과제 요건이 “별도 유료 라이선스 없이 실행 가능”입니다. 런타임 LLM은 키가 없으면 게임 자체가 죽습니다.
- 결정이 초당 수십 회입니다. 나사 하나에 손이 닿을 때마다 판단하므로 네트워크 왕복이 성립하지 않습니다. 측정된 배정 지연은 탭당 평균 0.012ms입니다.
- 색을 쓰는 코드는 틀리면 안 됩니다. 4.2의 분리 원칙에 따라, LLM은 정책을 설계하는 자리에 두고 **나사에 색을 쓰는 자리**에는 두지 않았습니다.

6.2 개발 단계의 AI 활용과 실제 지시

게임 규칙, 장르 선택, 스코프 결정은 사람이 내렸고, AI는 구현·계측·진단을 담당했습니다. 작업 방식에서 결정적이었던 지시는 두 가지입니다.

“UI를 만들기 전에 헤드리스로 엔진을 검증한다.”

보고만 하는 벤치가 아니라 **단언**하는 벤치를 만들게 했습니다. 이 한 가지가 여러 설계 결함을 화면에 닿기 전에 잡아냈습니다 (7.1).

“재미있는지는 짐작하지 말고 계측한다.”

src/probe/ 의 후보 비교, 이전 규칙을 폐기하게 만든 의사결정 깊이 진단, 그리고 7.1의 발견들이 전부 여기서 나왔습니다. 초기 시안은 **턴당 선택지 2.5개, 그중 58%가 최선수와 동점으로** 측정되어 폐기됐습니다 — 재미없다는 느낌이 아니라 수치로.

전체 진행은 저장소 커밋 기록에 남아 있습니다. 규칙을 여러 차례 갈아엎는 동안 **AI 총(추정기·글래스박스·벤치·페르소나)**은 **한 번도 재작성되지 않았**습니다 — 이 프로젝트의 자산이 어디에 있는지 보여주는 사실이라고 생각합니다.

7. 검증

npm run bench 는 봇 페르소나로 전체 게임을 **실제 tap()** 경로로 돌립니다. 심사자가 플레이하는 바로 그 코드에서 나온 수치입니다. 페르소나당 300런:

페르소나	해체한 벽	점수	탭	완성	외톨이 홀더	θ	탭당 ms
novice	17.10	43,959	594	196.5	1.16	0.543	0.018
casual	21.60	66,410	757	250.9	0.90	0.695	0.015
expert	18.47	47,063	643	212.9	0.62	0.835	0.026

- OK — 완결성 불변식이 모든 런의 모든 탭에서 유지됨
- OK — 900런이 **전부** 패배로 끝남. 탭 상한(12,000)에 걸린 런은 0개이므로 상한 인공물이 아닙니다
- OK — 실력이 보상됨: novice 17.10 < expert 18.47
- OK — θ 가 페르소나를 마땅한 순서로 정렬: 0.543 < 0.695 < 0.835

양 끝은 순서대로지만 casual이 가장 높습니다(21.60). 숨기지 않고 적습니다 — 실력 마진이 7%인 규칙에 난이도 적응을 걸면 이렇게 됩니다. 적응의 목적은 결과의 **분산을 줄이는** 것이지 순서를 뒤집는 것이 아니고, 지금은 뒤집히지 않는 선까지만 와 있습니다. 근본 해법은 적응을 더 줄이는 것이 아니라 **마진 자체를 넓히는** 것이며, 그 예산을 재는 도구가 7.2입니다.

7.1 계측이 잡아낸 것들 — 이번 규칙에서

증상	원인과 수정
한 턴에 손이 닿는 나사가 17.5개	아래 판의 볼트를 모서리 근처에 박아 두어, 그 모서리가 위에 쌓인 모든 판 밖으로 빠져나와 벽이 끝날 때까지 계속 닿았습니다. 열일곱 개 중에 고르는 것은 선택이 아닙니다. 볼트를 위 판의 footprint 안쪽으로 당겨 4.3개로 줄였습니다. 이 한 줄이 이번 작업에서 가장 중요한 수정입니다.
아무도 지지 않음 — 세 페르소나가 전부 3,000탭 상한 도달	구제 확률이 위험도 + (1-목표) 였는데, 빈 홀더가 0이면 위험도가 1로 포화해 누가 플레 이하든 구제가 확정 됐습니다. 두 항을 곱으로 바꿔, 위험도는 “도움이 얼마나 놓여 있는가”를 정하고 목표는 “그중 얼마를 받을 자격이 있는가”를 정하게 했습니다. 더불어 교착 자체가 패 배로 감지되지 않고 있었습니다 — 붓은 애초에 갈 곳 없는 나사를 고르지 않으므로.
솔버의 최선수를 따를수록 성적이 나빠짐	국면 평가가 “이 홀더를 끝낼 수 있는가”를 묻지 않았습니다. 벽에 같은 색이 하나도 안 남았는데 홀더를 그 색으로 잡그는 수를 태연히 최선수로 골랐습니다. 지원 항을 넣자 최선수가 무작위를 7%, 최악수를 2.3배 앞섭니다. 이걸 고치기 전의 후회(regret) 수치는 전부 의미가 없었습니다.
초보가 숙련자보다 더 멀리 감 (n=300에서 7.87 vs 6.97)	게이트가 목표 난이도의 임계값으로 풀리도록 되어 있어, 숙련자가 선을 넘는 순간 안전망이 한 계단에 사라졌습니다. 실력이 질벽 으로 도착한 것입니다. 게이트에서 θ 의존을 완전히 제거하고, 적응은 매끄러운 구제 항에만 남겼습니다.

7.2 실력 마진은 예산이다

위의 마지막 두 항목은 따로 재기 전까지 구분되지 않았습니다. “초보가 더 멀리 간다”는 관측의 자연스러운 해석은 “DDA가 과보호한다”이지만, **적응을 완전히 꺼도 순서는 그대로 뒤집혀 있었습니다.** 원인이 두 개였기 때문입니다.

그래서 `npm run margin` 은 배정기를 열려 놓고 **이 규칙 자체에서 실력이 얼마나 값어치가 있는지만** 잹니다:

최선수만 두기 9.67 벽
무작위 합법수 9.04 벽
최악수만 두기 4.22 벽

실력 마진 (최선수 / 무작위) 7%
순위 스프레드 (최선수 / 최악수) 2.29배
한 턴에 닿는 나사 4.3개 · 서로 다른 색 2.05종 · 완성 가능한 수가 있는 턴 48%

7%는 각주가 아니라 예산입니다. 숙련자의 난이도를 실력 마진보다 더 올리는 적응은, 아무리 원리가 옳아도 순위를 뒤집습니다. 그래서 정책 상수를 이 예산 **안쪽**으로 맞췄고 — 벤치가 그것을 확인합니다.

이것은 DDA를 논할 때 잘 이야기되지 않는 제약이라고 생각합니다. 적응의 정교함보다 **바탕 규칙의 실력 마진이 먼저**이고, 마진을 재지 않은 채 적응을 세게 걸면 “잘하는 사람이 손해 보는 게임”이 만들어집니다. 다음 작업은 적응을 줄이는 쪽이 아니라 **마진 자체를 넓히는 것**입니다 — 그러려면 1-ply 탐욕 평가에 선견(lookahead)이 필요합니다. 팔레트가 5종을 넘으면 지금 평가는 무작위에게 지기 시작하는데, 그 지점이 정확히 선견이 필요해지는 지점입니다.

8. 외부 에셋 및 오픈소스 출처

외부 에셋 없음. 이미지·사운드·모델·폰트·아이콘 파일이 저장소에 하나도 없습니다. 철판은 모서리를 둥글린 박스에 볼트 구멍을 파낸 기하이고, 나사는 육각 머리와 드라이버 소켓·와셔·샤프트를 조합한 기하입니다. 금속의 질감을 만드는 브러시드 러프니스

맵은 실행 시점에 캔버스에 5,000개의 가로선을 그려 만들고, 반사에 쓰는 환경광은 씬으로 생성합니다(HDRI 파일을 받지 않습니다). 서체는 사용자 기기의 시스템 폰트입니다.

패키지	라이선스	범위	용도
<code>three</code> ^0.185.1	MIT	dependency	3D 렌더링. 번들에 포함되는 유일한 런타임 의존성
<code>vite</code> ^6.0.5	MIT	devDependency	개발 서버 및 정적 빌드
<code>typescript</code> ^5.7.2	Apache-2.0	devDependency	타입 검사
<code>tsx</code> ^4.19.2	MIT	devDependency	벤치·프로브 실행
<code>@types/node</code>	MIT	devDependency	타입 정의
<code>@anthropic-ai/sdk</code>	MIT	devDependency	오프라인 도구용. 배포 번들에 미포함

빌드 산출물은 JS 553KB · CSS 8KB (gzip 144KB · 2.5KB)이고 **외부 요청이 0건**입니다. 대부분이 Three.js이며, 2D 캔버스 버전은 25KB였습니다 — 비주얼 품질을 위해 지불한 비용을 숨기지 않고 적어 둡니다.

9. 한계와 다음 단계

- **사람 데이터가 없습니다.** 정책은 봇 페르소나에 맞춰져 있습니다. 봇은 사람의 대용이지 사람이 아닙니다. 다음 단계는 실제 플레이 로그로 페르소나를 재적합하는 것입니다.
- **추정이 세션 내로 한정됩니다.** 계정·서버가 없으므로 0는 매 런 사전확률에서 다시 시작합니다. 세션 간 프로필을 유지하면 첫 30초의 정확도가 크게 올라갑니다. 해커톤 스코프에서 의도적으로 제외했습니다.
- **벽 기하는 아직 절차적이지 않습니다.** 층마다 분할 방향을 번갈아 두는 계단형 배치를 쓰고 있습니다. 기하까지 생성 대상으로 넣으면 “레벨 디자인 자동화”의 남은 절반이 채워집니다.
- **실력 마진이 7%로 좁습니다.** 7.2에서 전 값이고, 이 프로젝트에서 가장 중요한 미해결 항목입니다. 마진이 좁으면 적응의 여지도 그만큼 좁아집니다. 해법은 적응을 줄이는 것이 아니라 솔버에 선견을 넣어 규칙의 깊이를 드러내는 것입니다.
- **정책 상수는 손으로 정했습니다.** 봇 페르소나를 목적함수에 걸어 탐색하는 오프라인 튜너를 이전 규칙에서 구현해 30% 개선을 얻은 적이 있습니다. 다만 이 규칙에서는 튜너보다 마진을 넓히는 쪽이 먼저입니다 — 예산이 7%인 상태에서 정책을 미세 조정해 얻을 수 있는 것에는 한계가 있습니다.